

Input and Output in C

Standard Input and Output

Text Stream Model

- ▶ A text stream consists of a sequence of lines, each ending with a newline character.
- ▶ ANSI standard defines library functions that work on any system where C exists.
- ▶ Programs using standard library functions are portable across systems.

Simple Input Mechanism

getchar Function

```
int getchar(void);
```

- ▶ Reads one character at a time from standard input (keyboard).
- ▶ Returns the next input character or EOF on end of file.
- ▶ EOF is typically -1, but tests should use EOF for portability; it is a symbolic constant defined in stdio.h

Input Redirection

Substitute File for Keyboard Input

```
prog <infile
```

- ▶ File `infile` can replace standard input (keyboard).
- ▶ “`<infile`” is not included in the command-line arguments in `argv`.
- ▶ Similar behavior with pipes for input from other programs:
- ▶ `otherprog | prog` (e.g., `ls -l | grep "x"`)
- ▶ pipes the standard output of `otherprog` into the standard input for `prog`.

Simple Output Mechanism

putchar Function

```
int putchar(int);
```

- ▶ Puts the character on standard output (default: screen).
- ▶ Returns the character written or EOF on error.
- ▶ Output can be directed to a file: `prog >outfile`.

Output with printf

Formatted Output

- ▶ `printf` sends formatted output to standard output.
- ▶ `putchar` and `printf` output can be interleaved.

Header Inclusion

```
#include <stdio.h>
```

- ▶ Every source file using I/O library functions must include `<stdio.h>`.
- ▶ Enables proper compilation and linkage.

Example: Lowercasing Program

lower.c

```
#include <stdio.h>
#include <ctype.h>

int main() {
    int c;
    while ((c = getchar()) != EOF)
        putchar(tolower(c));
    return 0;
}
```


Standard I/O

- ▶ Standard I/O functions facilitate interaction with programs.
- ▶ Use `getchar`, `putchar`, and `printf` for simple cases.
- ▶ `#include <stdio.h>` ensures correct library usage.
- ▶ Enclosure in `<` and `>` implies a search is made for the header in a standard set of places
- ▶ On UNIX systems, typically in the directory `/usr/include`

Formatted Output - printf

Introduction

- ▶ The `printf` function translates internal values to characters.
- ▶ It converts, formats, and prints its arguments on the standard output under control of the format string.
- ▶ Returns the number of characters printed.

printf Function Signature

```
int printf(char *format, arg1, arg2, ...);
```

- ▶ `format` is the control string with conversion specifications.

Format String Components

- ▶ Ordinary characters: copied to the output stream.
- ▶ Conversion specifications: begin with %, ends with a conversion character.
- ▶ Format: `%[flags][minwidth][.][precision] [length] character`
- ▶ Fields enclosed in [] show that they are optional

Basic Printf Conversions

Character	Argument type; Printed As
d, i	int; decimal number
o	int; unsigned octal number (no leading 0)
x, X	int; unsigned hexadecimal number (no leading 0)
u	int; unsigned decimal number
c	int; single character
s	char*; print characters from the string
f	double; floating-point number
e, E	double; scientific notation
g, G	double; %e or %E if $\text{exp} < -4$ or $\geq \text{precision}$, else %f
p	void*; pointer representation
%	no argument; print a %

Format Modifiers and Flags

- ▶ Flags: - (left adjustment), + (sign), 0 (zero-padding), # (alternate forms: %#x to prefix 0x, %#o to prefix 0, %#f to force decimal point etc.)
- ▶ Width: Minimum field width, specified as a number.
- ▶ Precision: Maximum characters for strings, decimal places for floating-point.
- ▶ Length: h (short), l (long) if the integer is to be printed as a short, or a long.

Example Format Conversions

- ▶ `%10s`: width: $\max(10, \text{actual})$; if 10, right justified
- ▶ `%.10s`: 1st 10 char, or the whole if $\text{len} \leq 10$.
- ▶ `%-10s`: width: $\max(10, \text{actual})$; if 10, left justified.
- ▶ `%15.10s`: width: $\max(15, \text{actual})$; use leftmost $\min(10, \text{len})$.
- ▶ `%-15.10s`: as above. Left justified.

* width/precision

```
int width = 10;  
printf("|%*d|\n", width, 42);  
printf("|%-*s|\n", width, "Left-aligned");
```

- ▶ A width or precision may be specified as *, in which case the value is computed by converting the next argument (which must be an int).

Warning: printf

Argument Mismatch

- ▶ `printf` uses its first argument to decide the number and types of subsequent arguments.
- ▶ Mismatched arguments or types lead to incorrect results.
- ▶ Be cautious with format strings.

Example: Correct vs Incorrect

```
printf(s);          /* FAILS if s contains % */  
printf("%s", s); /* SAFE */
```

- ▶ Always use format strings to avoid undefined behavior.

sprintf Function

- ▶ `sprintf` writes into a string instead of standard output.
- ▶ `int sprintf(char *string, char *format, ...)` prints into the string according to the format.

Formatted Input - Scanf

```
int scanf(char *format, ...);
```

- ▶ The `scanf` function is the input analog of `printf`, providing similar conversion facilities in the opposite direction.
- ▶ `int scanf(char *format, ...)` reads characters from the standard input and interprets them based on the specified format.
- ▶ Results are stored through the remaining pointer arguments.
- ▶ The return value is the number of successfully matched and assigned input items.

sscanf Function

- ▶ `sscanf` reads from a string instead of standard input.
- ▶ `int sscanf(char *string, char *format, ...)` scans the string according to the format and stores resulting values through pointer arguments.

Format String Components

- ▶ Blanks or tabs: not ignored.
- ▶ Ordinary characters (not %): expected to match the next non-white space character.
- ▶ Conversion specifications: % [assignment suppression character *] [max width] [length] [conversion character].
- ▶ If * is present, the input field is skipped; no assignment is made.
- ▶ An input field is defined as a string of non-white space characters; extends to white space or for field width.
- ▶ White space: blank, tab, newline, carriage return, vertical tab, and formfeed.

Basic Scanf Conversions

Character	Input Data; Argument type
d	decimal integer; int*
i	integer; int*
o	octal integer; int*
u	unsigned decimal integer; unsigned int*
x	hexadecimal integer; int*
c	characters; char*
s	character string; char*
e, f, g	floating-point number; float*
%	literal % (no assignment)

Length Modifiers

- ▶ Prefix characters to conversions: h (short), l (long).
- ▶ Example: `%ld` for long integer.

Example: Rudimentary Calculator

```
#include <stdio.h>
main()
{
    double sum, v;
    sum = 0;
    while (scanf("%lf", &v) == 1)
        printf("\t%.2f\n", sum += v);
    return 0;
}
```

- Reads numbers from input and prints the running sum.

Example: Date Input

```
int day, year;  
char monthname[20];  
scanf("%d %s %d", &day, monthname, &year);
```

- Reads dates in the format 25 Dec 1988.

Example: Reading Dates with Different Formats

```
while (getline(line, sizeof(line)) > 0) {  
    if (sscanf(line, "%d %s %d", &day, monthname, &year) == 3)  
        printf("valid: %s\n", line);  
    else if (sscanf(line, "%d/%d/%d", &month, &day, &year) == 3)  
        printf("valid: %s\n", line);  
    else  
        printf("invalid: %s\n", line);  
}
```

- Handles different date formats in input lines.

typedef

typedef in C

- ▶ C provides typedef for creating new data type names.
- ▶ `typedef int Length;` `Length` is a synonym for `int`.
- ▶ `typedef char *String;` `String` is a synonym for `char *`.
- ▶ Used in declarations, casts, etc., just like the original types.

Example - Length

```
typedef int Length;
```

```
Length len, maxlen;
```

```
Length *lengths[];
```

- ▶ Length is now a synonym for int.

Example - String

```
typedef char *String;  
  
String p, lineptr[MAXLINES], alloc(int);  
int strcmp(String, String);  
  
p = (String) malloc(100);
```

- String is now a synonym for char *.

Example - Tree Nodes

```
typedef struct tnode *Treeptr;  
  
typedef struct tnode {  
    char *word;  
    int count;  
    struct tnode *left;  
    struct tnode *right;  
} Treenode;
```

- ▶ Creating typedefs for tree nodes.
- ▶ Treeptr is a pointer to Treenode.

Example - talloc with Typedefs

```
Treeptr talloc(void)
{
    return (Treeptr) malloc(sizeof(Treenode));
}
```

- ▶ talloc now uses the Treeptr typedef.

typedef: Key Points

- ▶ typedef does not create a new type; it adds a new name for an existing type.
- ▶ Improves program documentation and readability.
- ▶ Useful for parameterizing programs against portability issues. If used for machine-dependent types, only the typedefs need change when the program is moved. E.g., use typedef names for various integer quantities, then change typedefs using short, int, or long as appropriate for each host machine.
- ▶ Can be used to create complex type names, enhancing code readability.